Octobre 2026

Rapport de TP 1

Isolation et confinement avancés

SARR Pape Aldiouma

L3 Cybersécurité · Module : Sécurité des systèmes avancée

Rapport de travaux pratiques : TP 1, Isolation et confinement avancés. Suite du rapport « Segmentation & Hardening » sur le même laboratoire SenTech Solutions : Docker, AppArmor, Trivy, Falco.





République du Sénégal, Un Peuple, Un But, Une Foi
Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation
Direction Générale de l'Enseignement Supérieur Privé
Institut Supérieur de Management
Année académique 2026-2027

SOMMAIRE

Partie I : Présentation du travail..... 3

Contexte et objectifs..... 3
Environnement du laboratoire..... 3
Méthode de travail et conventions du rapport..... 4
Points de vigilance propres à notre laboratoire..... 5

Partie II : TP 1 : Isolation et confinement avancés..... 6

Partie 0 : installation de Docker..... 6
Partie 1 : du DAC au MAC, pourquoi confiner au-delà des permissions..... 8
Partie 2 : décomposer les pouvoirs de root, les capabilités..... 9
Partie 3 : filtrer les appels système, seccomp..... 12
Partie 4 : les briques de l'isolation, namespaces et cgroups..... 13
Partie 5 : le contrôle d'accès obligatoire avec AppArmor..... 16
Partie 6 : sécurité des conteneurs Docker, assembler les mécanismes..... 19
Partie 7 : hygiène des images, analyse avec Trivy..... 25
Partie 8 : comprendre pour prévenir, l'évasion de conteneur..... 29
Partie 9 : détecter l'intrusion à l'exécution, Falco..... 31
Partie 10 : synthèse, checklist et fiche finale du TP 1..... 33

Partie I : Présentation du travail

Contexte et objectifs

Ce rapport fait suite au rapport « Segmentation & Hardening », dans lequel nous avons construit le laboratoire de SenTech Solutions : un réseau segmenté derrière un pare-feu OPNsense, un serveur Debian durci (SSH par clé sur le port 2222, pare-feu iptables en DROP, Fail2Ban, Auditd, AIDE, ACL) et des sauvegardes. Le présent document réalise, sur exactement le même laboratoire, le travail pratique « TP 1, Isolation et confinement avancés » du module « Sécurité des systèmes avancée » :

- **TP 1, Isolation et confinement avancés :** décomposer les pouvoirs de root (capabilités), filtrer les appels système (seccomp), comprendre les briques d'un conteneur (namespaces et cgroups), appliquer un contrôle d'accès obligatoire (AppArmor), construire un conteneur Docker durci, contrôler les images (Trivy), reproduire puis neutraliser une évasion de conteneur, et détecter l'intrusion à l'exécution (Falco).

La question du directeur de SenTech guide tout le TP : « si un attaquant compromet une application dans un conteneur, peut-il atteindre le serveur et les autres services ? ». Le rapport 1 (« Segmentation & Hardening ») a fermé les portes du serveur ; ce TP 1 enferme chaque processus qui tourne dessus.

Note sur le rapport TP 2 : le travail pratique « Ingénierie de détection et chasse aux menaces » (MITRE ATT&CK, Sigma, YARA, MISP), mené sur le même laboratoire, fait l'objet d'un rapport séparé et autonome. Les deux rapports se lisent indépendamment l'un de l'autre.

Environnement du laboratoire

Toutes les machines sont des machines virtuelles QEMU/KVM gérées par libvirt et virt-manager, sur un hôte Ubuntu (HP EliteBook 840 G3, 16 Go de RAM). Elles partagent le même réseau virtuel isolé lan-sentech (192.168.10.0/24), dont OPNsense est la passerelle.

Machine	Adresse	Système	Rôle dans le TP
OPNsense	192.168.10.1	OPNsense	Pare-feu, NAT, DNS (Unbound) ; périmètre du laboratoire
Kali Linux	192.168.10.99	Kali	Poste d'observation et de tests réseau
Debian Serveur	192.168.10.10	Debian 13 « Trixie »	Machine durcie du rapport 1 ; héberge Docker, AppArmor, Falco ; cible de toutes les manipulations
Debian Client	192.168.10.11	Debian	Non utilisé dans ce TP (reste éteint)

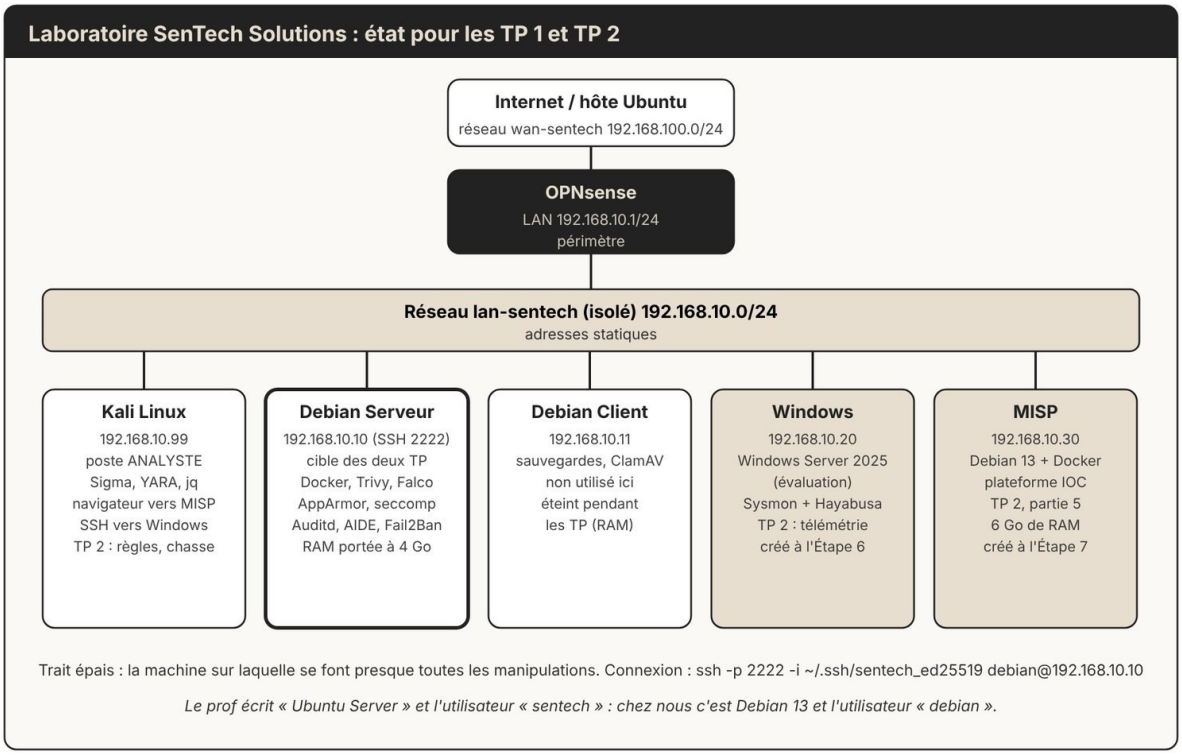


Figure 1 : Le laboratoire SenTech étendu pour le TP 1 : le serveur est la cible, Kali l'observateur

Méthode de travail et conventions du rapport

Chaque manipulation suit le même plan, afin que le lecteur puisse la rejouer et la vérifier :

1. **Objectif :** ce que l'on cherche à démontrer ou à mettre en place ;
2. **Principe :** l'explication du concept ou de la commande, avec nos mots ;
3. **Procédure :** les commandes exactes, précédées de l'étiquette de la machine où elles se tapent (« Sur Debian Serveur », « Sur Kali Linux ») ;
4. **Résultat attendu :** ce que la sortie doit contenir pour que la manipulation soit réussie ;
5. **Analyse :** ce que le résultat prouve ;
6. **Capture :** un cadre gris numéroté indique la capture d'écran à insérer, avec la liste de ce qu'elle doit montrer.

> **REMARQUE** **Comment utiliser les cadres de capture**
>
> Chaque cadre « CAPTURE n » est vide : remplacez-le par votre capture (terminal complet, avec l'invite visible, pour que la machine soit identifiable). Les chiffres cités dans le texte (PID, nombre de CVE, horodatages, scores) sont des exemples : après insertion de la capture, vérifiez que le texte correspond à ce que vous voyez et corrigez les chiffres si besoin. L'index de toutes les captures figure à la fin du document.

Quatre habitudes ont été appliquées partout et sont justifiées dans le texte :

- **Toujours sudo devant docker :** l'utilisateur debian n'appartient pas au groupe docker, car ce groupe donne un accès root de fait au serveur (principe du moindre privilège du rapport 1).
- **Toujours sudo devant les commandes d'administration** (getcap, aa-status, sysctl, iptables) : sur Debian 13, le PATH d'un utilisateur ordinaire ne contient pas /usr/sbin.
- **Deux terminaux ouverts :** un sur le serveur (SSH depuis Kali, port 2222), un sur Kali ; on évite plus de quatre nouvelles connexions SSH par minute, sinon la règle SSH_SCAN du rapport 1 bannit l'adresse.

- **Un état de départ récupérable :** instantané avant chaque phase risquée.

Points de vigilance propres à notre laboratoire

Le sujet de TP a été écrit pour un laboratoire légèrement différent du nôtre. Les écarts suivants ont été identifiés avant de commencer et expliquent plusieurs adaptations de commandes dans la suite du rapport.

Consigne du sujet	Notre laboratoire	Conséquence
Hyperviseur VMware, « Snapshot »	QEMU/KVM, libvirt	Instantanés avec `virsh snapshot-create-as`
Ubuntu Server, utilisateur sentech	Debian 13, utilisateur debian	Chemins /home/sentech/… devenus /home/debian/…
Pas de pare-feu particulier	iptables INPUT et FORWARD en DROP, ip_forward = 0	Docker modifie les deux : à constater puis à remettre en ordre
Journaux AppArmor lus avec dmesg	Auditd actif	Les refus AppArmor sont dans /var/log/audit/audit.log
umask 022 implicite	umask 027 (rapport 1)	Fichiers créés en 640 : impact sur COPY Docker, clés GPG, cron.d

Partie II : TP 1 : Isolation et confinement avancés

Fil conducteur. Le noyau Linux offre quatre mécanismes complémentaires pour restreindre un processus. Un conteneur n'est que leur assemblage : ce n'est pas une petite machine virtuelle, c'est un processus ordinaire du serveur, enfermé dans une cage faite de ces quatre pièces.

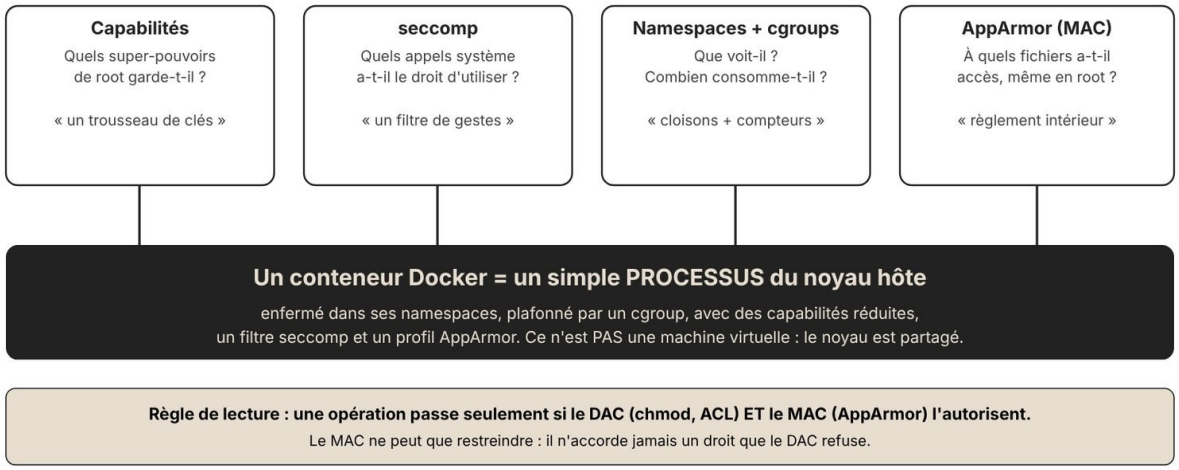


Figure 2 : Les quatre mécanismes de confinement du noyau, et le conteneur qui les assemble

Mécanisme	Question à laquelle il répond	Image mentale
Capabilités	Quels super-pouvoirs de root ce processus a-t-il ?	Un trousseau où chaque clé ouvre une seule porte, au lieu d'un passe-partout
seccomp	Quels appels système peut-il utiliser ?	Un filtre qui n'autorise que certains gestes
Namespaces et cgroups	Que voit-il ? Combien consomme-t-il ?	Des cloisons (ce qu'il voit) et des compteurs (ce qu'il consomme)
MAC (AppArmor)	À quels fichiers et opérations a-t-il accès, quelle que soit son identité ?	Un règlement intérieur qui s'impose à tous, directeur compris

Partie 0 : installation de Docker

Objectif. Installer le moteur de conteneurs du TP et constater ce qu'il change sur un serveur déjà durci.

Principe. `docker.io` est le paquet Debian du moteur Docker. `enable --now` démarre le service tout de suite et à chaque démarrage. `docker version` doit afficher une partie « Client » et une partie « Server » : si le serveur manque, le service ne tourne pas. L'option `--rm` de `hello-world` supprime le conteneur après exécution.

Procédure.

Sur Debian Serveur

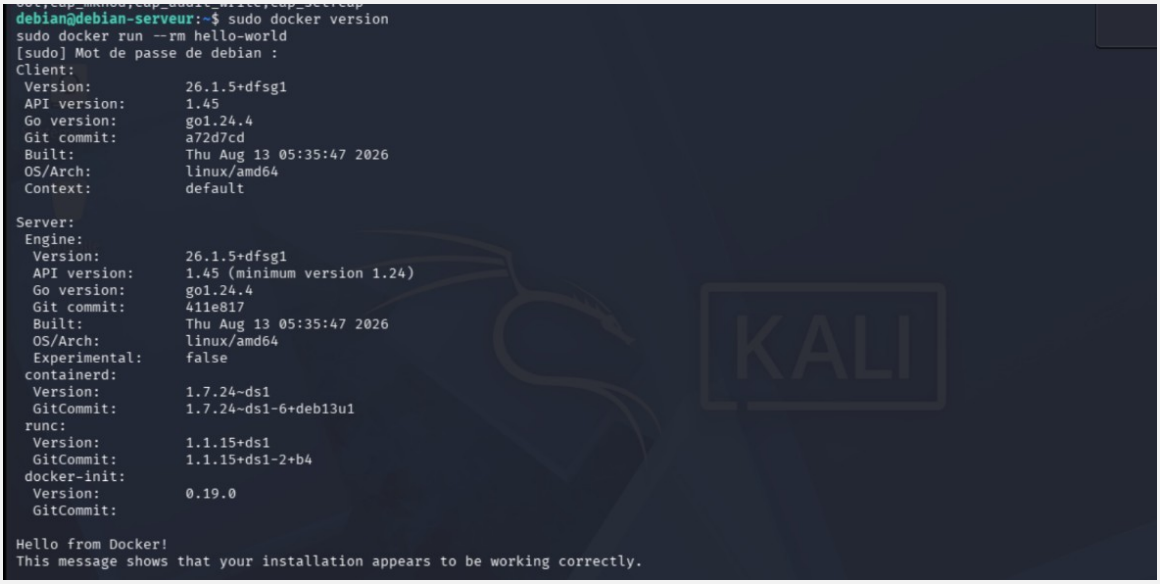
```
sudo apt update
sudo apt install -y docker.io
```

```
sudo systemctl enable --now docker

sudo docker version
sudo docker run --rm hello-world
```

CAPTURE 1 : Docker installé et hello-world

À montrer : sudo docker version (parties Client et Server) et le message Hello from Docker!.



Pourquoi `sudo docker` et non l'ajout au groupe `docker` ? Le groupe docker donne la prise de contrôle d'un moteur qui tourne en root : un membre peut lancer `docker run -v /:/host alpine` et lire ou modifier tout le disque. C'est un accès root déguisé. Par cohérence avec le principe du moindre privilège du rapport 1, debian n'est pas dans ce groupe.

DIFFICULTÉ **Des outils complémentaires, demandés par le sujet, installés dès cette étape**

Le sujet demande, en plus de Docker, `sudo apt install -y libcap2-bin apparmor-utils gcc stress-ng jq`. Ces paquets ne servent pas encore à ce stade mais conditionnent toute la suite : `libcap2-bin` fournit `getcap`/`setcap` (Partie 2), `apparmor-utils` fournit `aa-status`/`aa-exec`/`aa-enforce` (Partie 5), `gcc` compile le petit serveur C `bind80` (Partie 2.3), `stress-ng` provoque la surcharge mémoire du test de cgroup (Partie 4.3) et `jq` lit les résultats JSON de Trivy (Partie 7). Nous les avons installés ici, en une seule fois, plutôt que de les redemander au fil du rapport :

Sur Debian Serveur

```
sudo apt install -y libcap2-bin apparmor-utils gcc stress-ng jq
```

Vérification de la configuration de sécurité de Docker et de son effet sur le serveur.

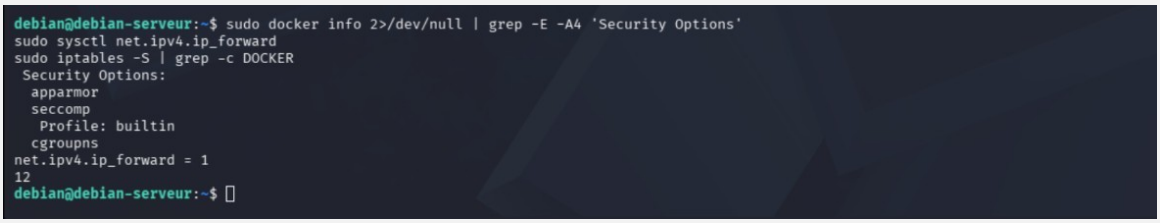
Sur Debian Serveur

```
sudo docker info 2>/dev/null | grep -E 'Server Version|Cgroup Version|Security
Options|Storage Driver'
sudo iptables -S | grep '^-P'
sysctl net.ipv4.ip_forward
```

Résultat attendu. `Cgroup Version: 2` ; des options de sécurité contenant `apparmor`, `seccomp` (profil builtin) et `cgroupns` ; les politiques INPUT DROP et FORWARD ; et `net.ipv4.ip_forward = 1`.

Analyse. Dès son démarrage, Docker (1) crée ses propres chaînes iptables (DOCKER, DOCKER-USER…), (2) active le routage IP (`ip_forward = 1`) alors que notre durcissement l'avait mis à 0, (3) crée le réseau `docker0` en 172.17.0.0/16. Deux règles en découlent pour toute la suite : ne jamais lancer `netfilter-persistent save` tant que Docker tourne (il enregistrerait les chaînes de Docker dans `rules.v4`), et remettre l'état durci à la fin. La sauvegarde propre existe déjà : `/root/baseline_TP/rules.v4.avant_TP`.

CAPTURE 2 : Docker et les protections du noyau



Partie 1 : du DAC au MAC, pourquoi confiner au-delà des permissions

1.1 La limite du modèle DAC

Dans le rapport 1 nous avons utilisé le DAC (contrôle d'accès discrétionnaire) : le propriétaire d'un fichier décide qui le lit ou l'écrit (chmod, chown, ACL). Ce modèle a deux angles morts :

- **root ignore le DAC :** l'utilisateur root (uid 0) contourne toutes les permissions, grâce à la capability CAP_DAC_OVERRIDE ;
- **le DAC raisonne par identité, pas par comportement :** il dit qui peut agir, jamais ce qu'un programme donné a le droit de faire. Un serveur web n'a aucune raison de lire /etc/shadow, mais le DAC ne l'interdit pas s'il tourne avec des droits suffisants.

1.2 L'idée du MAC

Le MAC (contrôle d'accès obligatoire) ajoute une politique fixée par l'administrateur que même root ne peut pas contourner. On ne demande plus « qui es-tu ? » mais « quel programme es-tu, et que peux-tu faire ? ». Règle de lecture : une opération est autorisée seulement si le DAC ET le MAC l'autorisent ; le MAC ne peut que restreindre.

1.3 Démonstration : root ignore le DAC

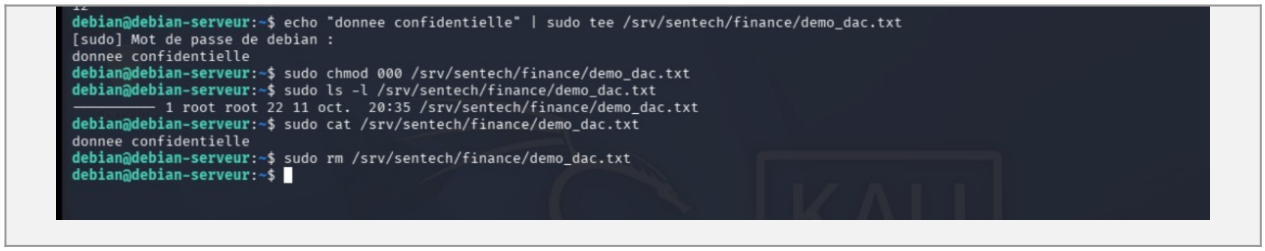
Procédure.

Sur Debian Serveur

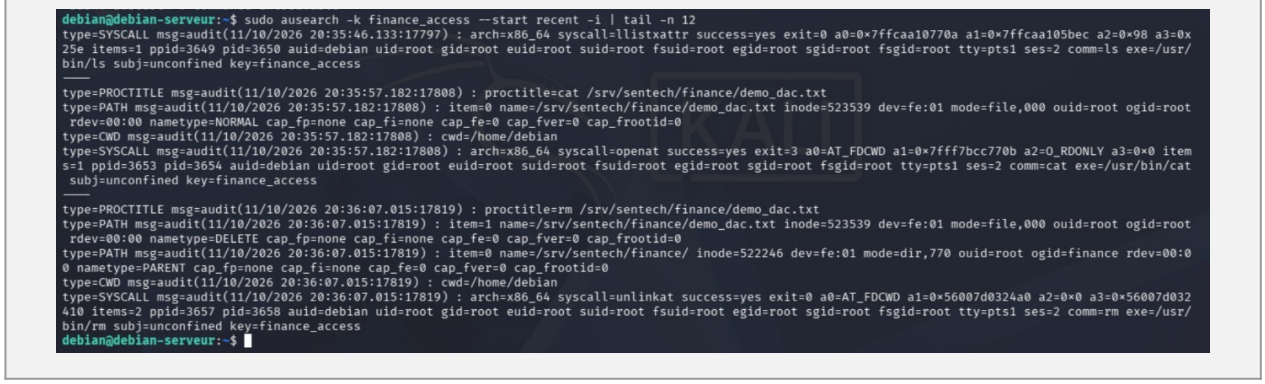
```
echo "donnee confidentielle" | sudo tee /srv/sentech/finance/demo_dac.txt
sudo chmod 000 /srv/sentech/finance/demo_dac.txt
sudo ls -l /srv/sentech/finance/demo_dac.txt
sudo cat /srv/sentech/finance/demo_dac.txt
sudo rm /srv/sentech/finance/demo_dac.txt
```

Analyse. Le fichier a la permission `000` (aucun droit pour personne) et pourtant `sudo cat` l'affiche : root passe à travers le DAC. La règle Auditd `finance_access` du rapport 1 a journalisé ces accès (`sudo ausearch -k finance_access --start recent -i | tail -n 12`).

CAPTURE 3 : Root ignore le DAC



CAPTURE 4 : Auditd a tracé l'accès



Partie 2 : décomposer les pouvoirs de root, les capabilités

2.1 Le concept

Historiquement, un processus est soit root (tous les pouvoirs), soit ordinaire (aucun pouvoir privilégié). Les capabilités découpent le pouvoir de root en une quarantaine de privilèges indépendants : on peut donner un seul privilège à un programme au lieu de root entier.

Capability	Pouvoir accordé
CAP_NET_BIND_SERVICE	Ouvrir un port réseau inférieur à 1024
CAP_NET_RAW	Forger des paquets bruts (ce que fait ping)
CAP_NET_ADMIN	Configurer le réseau (interfaces, pare-feu)
CAP_SYS_ADMIN	« Fourre-tout » très large : monter des systèmes de fichiers, créer des namespaces. À éviter
CAP_CHOWN	Changer le propriétaire d'un fichier
CAP_DAC_OVERRIDE	Ignorer les permissions de lecture et d'écriture
CAP_SYS_PTRACE	Inspecter ou déboguer la mémoire d'autres processus

Chaque processus possède cinq ensembles de capabilités (Permitted, Effective, Inheritable, Bounding, Ambient). On retient surtout Effective (CapEff) : les pouvoirs réellement actifs à l'instant présent.

2.2 Observer les capabilités

Procédure.

Sur Debian Serveur

```
sudo getcap "$(command -v ping)"
ls -l "$(command -v ping)"

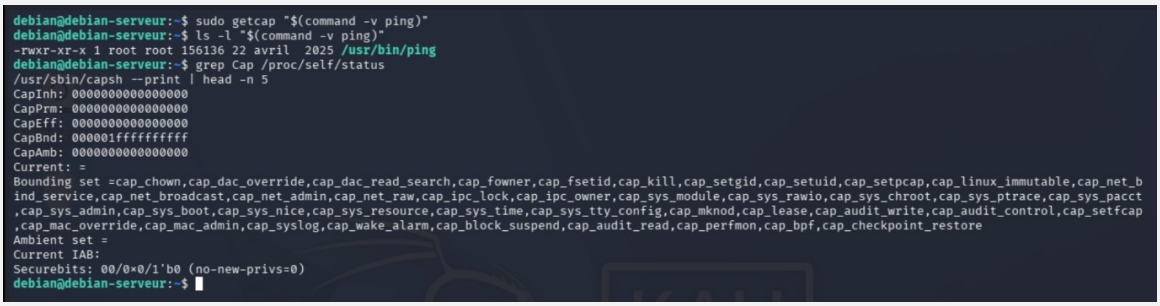
grep Cap /proc/self/status
/usr/sbin/capsh --print | head -n 5

sudo grep Cap /proc/self/status

/usr/sbin/capsh --decode=0000000000000400
/usr/sbin/capsh --decode=00000000a80425fb
```

Analyse. Sur Debian 13, `ping` n'a pas toujours de capability : le noyau autorise aussi `ping` sans privilège via le réglage `net.ipv4.ping_group_range`. Dans ce cas `getcap` ne renvoie rien et `ls -l` montre un fichier sans le `s` d'un SUID. La leçon reste la même : `ping` n'est plus un SUID root.

CAPTURE 5 : Observation des capabilités



2.3 Manipulation : ouvrir le port 80 sans être root

Problème métier. Un service SenTech doit écouter sur le port 80, réservé à root (ports inférieurs à 1024). Plutôt que de lancer tout le service en root, on lui donne uniquement CAP_NET_BIND_SERVICE.

Étape 1, créer le petit serveur en C. Le programme crée une « prise » réseau (socket), tente de la lier au port 80 (bind), se met en écoute (listen) puis attend (pause). C'est bind qui exigera la capability.

Sur Debian Serveur

```
cat > ~/bind80.c <<'EOF'
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main(void) {
  int s = socket(AF_INET, SOCK_STREAM, 0);
  struct sockaddr_in a;
  memset(&a, 0, sizeof a);
  a.sin_family = AF_INET;
  a.sin_port = htons(80);
  a.sin_addr.s_addr = INADDR_ANY;
  if (bind(s, (struct sockaddr *)&a, sizeof a) < 0) { perror("bind"); return
1; }
  if (listen(s, 1) < 0) { perror("listen"); return 1; }
  printf("Ecoute sur le port 80 (PID %d) - Ctrl+C pour quitter\n", getpid());
  pause();
  return 0;
}
```

```
EOF

gcc -O2 -o ~/bind80 ~/bind80.c
```

Étape 2, Test 1 sans capability (utilisateur ordinaire) :

Sur Debian Serveur

```
~/bind80
```

Étape 3, accorder la capability ciblée et la contrôler. Les guillemets simples empêchent le shell d'interpréter `+` et `=`.

Sur Debian Serveur

```
sudo setcap 'cap_net_bind_service=+ep' ~/bind80
sudo getcap ~/bind80
```

Étape 4, Test 2 avec la capability, et preuve que le processus n'est pas root (lancé en arrière-plan avec &) :

Sur Debian Serveur

```
~/bind80 &
sleep 1
ss -tlnp 'sport = :80'
ps -o user,pid,cmd -C bind80
grep CapEff /proc/$(pgrep -x bind80)/status
/usr/sbin/capsh --decode=$(grep CapEff /proc/$(pgrep -x bind80)/status | awk
'{print $2}')
```

Étape 5, arrêter le serveur et retirer la capability (on ne laisse pas un pouvoir qui n'a plus d'utilité) :

Sur Debian Serveur

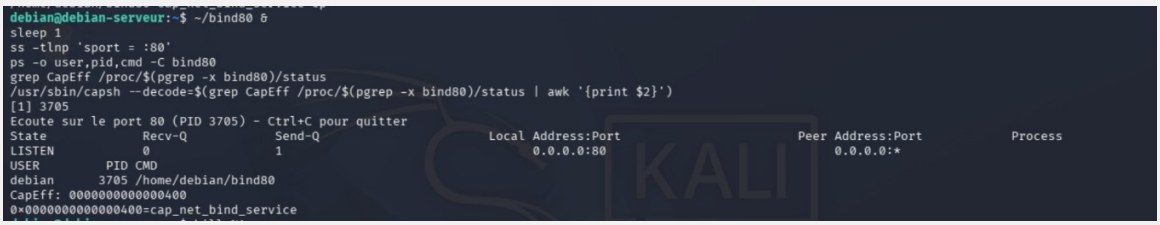
```
kill %1
sudo setcap -r ~/bind80
sudo getcap ~/bind80
```

Analyse. Le programme écoute sur le port 80, son propriétaire est `debian` (pas root), et son unique pouvoir actif est `cap_net_bind_service` : c'est le principe du moindre privilège en action. Le test est local : aucune règle de pare-feu n'est modifiée, et depuis Kali le port 80 resterait `filtered`, ce qui est voulu. Recompiler `bind80` efface la capability (elle est attachée au fichier) : il faudrait refaire le `setcap`.

CAPTURE 6 : Test 1 : sans capability



CAPTURE 7 : Test 2 : avec la capability ciblée



CAPTURE 8 : Capability retirée

À montrer : getcap ~/bind80 ne renvoyant plus rien.



Pourquoi `CAP_NET_BIND_SERVICE` est plus sûr que root. Si le service est compromis, l'attaquant n'hérite que de la possibilité d'ouvrir un port inférieur à 1024, alors que root lui donnerait tous les pouvoirs : lire /etc/shadow, charger des modules noyau, monter des disques, modifier le pare-feu. On limite les dégâts d'une compromission au strict nécessaire.

La capability la plus dangereuse du tableau est CAP_SYS_ADMIN : fourre-tout qui regroupe des dizaines d'opérations (monter des systèmes de fichiers, créer des namespaces, configurer des périphériques) et sert de point de départ à de nombreuses évasions de conteneur ; l'accorder revient presque à donner root.

Partie 3 : filtrer les appels système, seccomp

3.1 Le concept

Tout ce qu'un programme demande au noyau (ouvrir un fichier, créer un processus, monter un disque, se connecter) passe par un appel système (syscall). Un programme légitime n'en utilise qu'une petite fraction. seccomp (secure computing mode) définit une liste de syscalls autorisés ou interdits : tout appel interdit est refusé (erreur EPERM) ou tue le processus. Beaucoup d'attaques (évasion, escalade) reposent sur des syscalls rarement utiles à une application normale (mount, ptrace, unshare, keyctl) : les interdire réduit la surface d'attaque du noyau.

3.2 Manipulation : confiner un shell avec systemd

systemd applique un filtre seccomp à un service (option SystemCallFilter). systemd-run permet de le démontrer à chaud : on lance un shell auquel on retire la famille de syscalls @mount.

Procédure.

Sur Debian Serveur

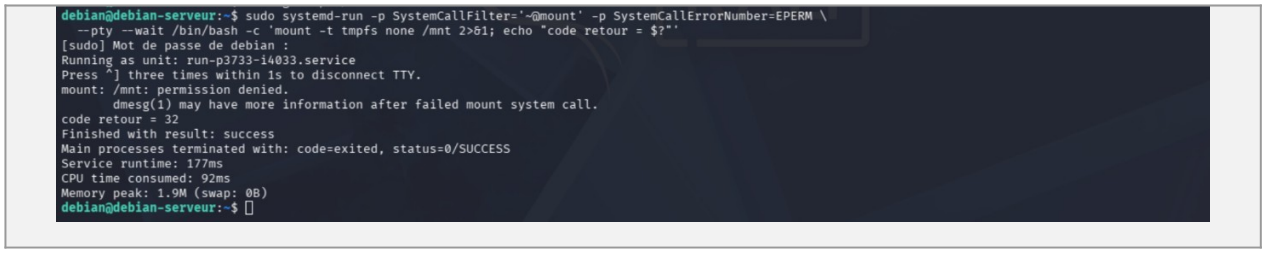
```
sudo systemd-run -p SystemCallFilter='~@mount' -p SystemCallErrorNumber=EPERM \
 --pty --wait /bin/bash -c 'mount -t tmpfs none /mnt 2>&1; echo "code retour =
$?"'
```

Décodage. `~` signifie « interdire », `@mount` est la famille des syscalls de montage, `SystemCallErrorNumber=EPERM` fait renvoyer « opération non permise » au lieu de tuer le processus, `--pty --wait` affiche la sortie et attend la fin. Les guillemets simples protègent `$?` : il est évalué dans le shell lancé.

Analyse. Le montage échoue alors même que la commande tourne en root. La commande `mount` traduit l'erreur EPERM par « permission denied » avec le code retour 32 (échec de montage) ; le sujet annonçait « Operation not permitted », mais ce qui compte est l'échec du montage et le code non nul. Autres familles utiles : @reboot, @swap, @module, @raw-io, @privileged, @debug, @obsolete.

Lien avec le rapport 1 : la règle Auditd `mount_ops` surveille le syscall mount (`sudo ausearch -k mount_ops --start recent -i | tail -n 8`), sans garantie de résultat selon la façon dont le noyau journalise un appel filtré.

CAPTURE 9 : seccomp interdit mount, même à root



3.3 Mesurer l'exposition d'un service

systemd-analyze security note le niveau de confinement d'un service : chaque protection absente ajoute des points d'exposition (même état d'esprit que Lynis, appliqué service par service).

Sur Debian Serveur

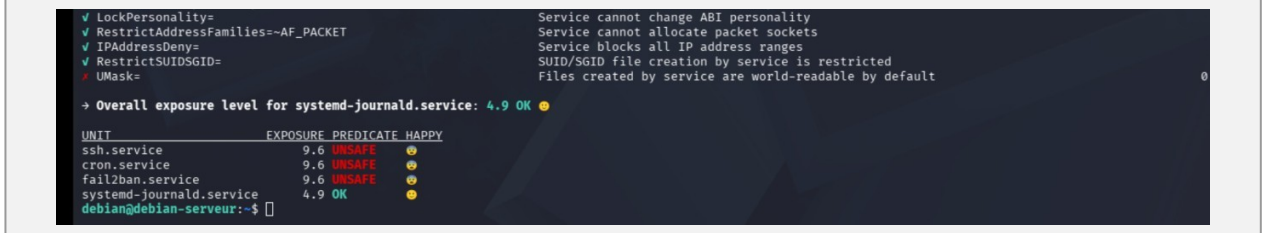
```
systemd-analyze security --no-pager ssh.service
systemd-analyze security --no-pager ssh.service cron.service fail2ban.service
systemd-journald.service
```

Lecture du score. De 0 (très confiné) à 10 (aucune protection) : OK jusqu'à 2, puis MEDIUM, EXPOSED, UNSAFE (au-delà de 7,0). Un service qui n'a jamais été conçu pour être confiné (ssh, cron) obtient souvent 9 à 10 : ce n'est pas un défaut de notre configuration, c'est son état d'origine. Un service moderne comme systemd-journald obtient un score nettement meilleur. L'outil indique où améliorer (ProtectSystem=strict, NoNewPrivileges=yes, PrivateTmp=yes).

> **ATTENTION** **Ne pas modifier ssh.service pour améliorer son score**
>
> Ajouter des restrictions à ssh.service sans test peut couper l'accès au serveur.

CAPTURE 10 : Exposition des services

À montrer : Le tableau comparatif de la seconde commande, avec les scores de ssh, cron, fail2ban et systemd-journald.



Deux familles de syscalls qu'un serveur web n'a aucune raison d'utiliser : @mount (monter ou démonter des systèmes de fichiers) et @reboot (redémarrer la machine) ; on pourrait ajouter @module, @swap ou @raw-io. Un serveur web sert des pages : il ne monte pas de disque et ne redémarre pas le système.

Partie 4 : les briques de l'isolation, namespaces et cgroups

4.1 Le concept

Deux mécanismes du noyau donnent l'illusion d'« ordinateurs séparés » sur une même machine :

- **Namespaces :** ce qu'un processus voit. Chaque type isole une ressource : PID (les processus), NET (le réseau), MNT (les montages), UTS (le nom d'hôte), IPC, USER (les identités), CGROUP.

- **cgroups :** ce qu'un processus consomme. Ils limitent et comptabilisent CPU, mémoire, entrées-sorties, nombre de processus.

Un conteneur n'est donc pas une machine virtuelle : c'est un processus ordinaire placé dans ses propres namespaces et dans un cgroup, complété par des capabilités réduites, un filtre seccomp et un profil AppArmor. Le noyau est partagé avec l'hôte : c'est pourquoi un conteneur est léger, et pourquoi son isolation est plus fragile que celle d'une machine virtuelle.

4.2 Manipulation : fabriquer un « conteneur » avec unshare

Isolation des processus (namespace PID). On compare l'identifiant du namespace avant et dans un nouveau namespace.

Sur Debian Serveur

```
readlink /proc/self/ns/pid
sudo unshare --pid --fork --mount-proc /bin/bash
```

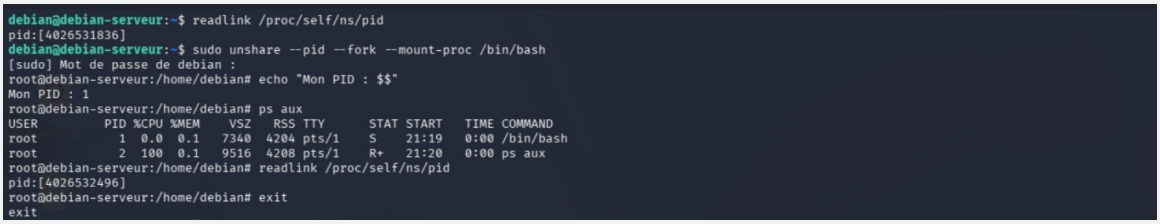
Dans le nouveau namespace

```
echo "Mon PID : $$"
ps aux
readlink /proc/self/ns/pid
exit
```

Décodage. --pid crée le namespace PID ; --fork est obligatoire (le premier processus du namespace doit être un fils, qui devient le PID 1) ; --mount-proc remonte /proc dans le nouvel espace, sinon ps lirait encore les processus de l'hôte.

Analyse. `$$` vaut 1 et `ps aux` ne montre que `bash` et `ps` : le shell se croit seul au monde alors que le serveur exécute des dizaines de processus. Les deux numéros `pid:[...]` différents prouvent que ce sont deux namespaces distincts.

CAPTURE 11 : Namespace PID



Isolation du nom d'hôte (namespace UTS), liste des namespaces, réseau vide.

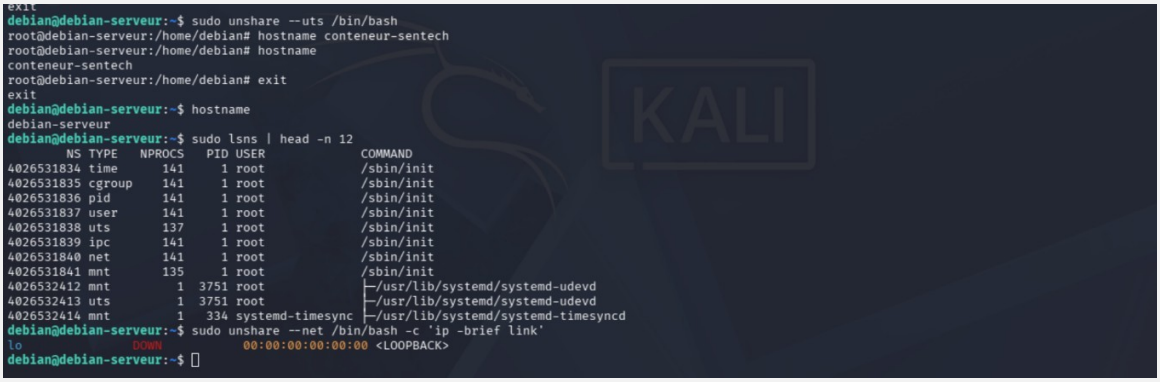
Sur Debian Serveur

```
sudo unshare --uts /bin/bash
hostname conteneur-sentech
hostname
exit
hostname

sudo lsns | head -n 12
sudo unshare --net /bin/bash -c 'ip -brief link'
```

Analyse. Le deuxième `hostname` affiche `conteneur-sentech` (uniquement dans la cage) ; après `exit`, le vrai nom est inchangé. `lsns` doit être lancé avec sudo pour lister tous les namespaces du système. Dans le namespace réseau vide, seule l'interface lo (éteinte) apparaît : le processus n'a plus accès à aucune carte réseau de l'hôte. Il ne faut pas oublier `exit` après `unshare ... /bin/bash` : on est dans la cage, et toute commande y est exécutée.

CAPTURE 12 : Namespaces UTS, lsns et réseau



4.3 Manipulation : plafonner les ressources avec un cgroup

Debian 13 utilise cgroup v2.

Sur Debian Serveur

```
cat /sys/fs/cgroup/cgroup.controllers
sudo systemd-run -p MemoryMax=64M -p MemorySwapMax=0 --pty --wait /bin/bash
```

Dans le shell plafonné

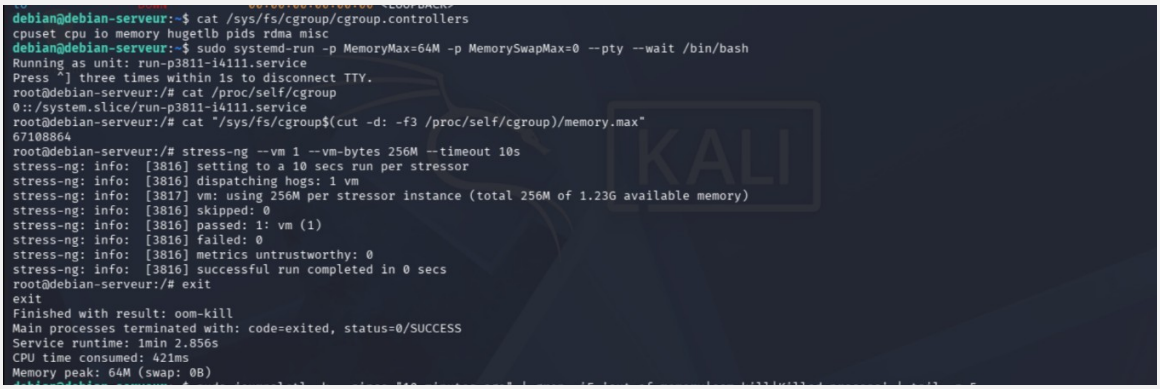
```
cat /proc/self/cgroup
cat "/sys/fs/cgroup$(cut -d: -f3 /proc/self/cgroup)/memory.max"
stress-ng --vm 1 --vm-bytes 256M --timeout 10s
exit
```

Sur Debian Serveur

```
sudo journalctl -k --since "10 minutes ago" | grep -iE 'out of memory|oom-kill|
Killed process' | tail -n 5
```

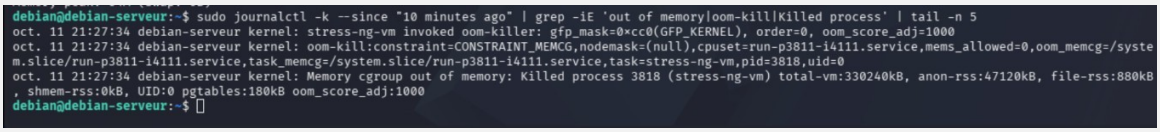
Analyse. Le mot `memcg` signale que c'est le cgroup (et non la mémoire totale de la machine) qui a déclenché l'arrêt : la limite est imposée par le noyau, pas seulement suggérée. Nous avons utilisé `systemd-run` en mode service (`--pty --wait`) plutôt que `--scope --pty` du sujet, incompatible sur plusieurs versions de systemd ; le résultat est identique. Les valeurs (256 Mo pour une limite de 64 Mo) ne doivent pas être augmentées « pour voir » : sans cgroup, on épuiserait la mémoire du serveur et l'OOM-killer global pourrait tuer sshd ou auditd.

CAPTURE 13 : Limite mémoire du cgroup



CAPTURE 14 : Preuve de l'OOM-kill

À montrer : La ligne Memory cgroup out of memory: Killed process … du journal du noyau.



Ce qu'est techniquement un conteneur. Un conteneur est un processus Linux ordinaire. Le noyau l'isole avec des namespaces (il ne voit que ses propres processus, son réseau, ses montages) et limite ses ressources avec un cgroup. On y ajoute capabilités réduites, filtre seccomp et profil AppArmor : ce n'est pas une machine virtuelle, le noyau est partagé avec l'hôte.

Partie 5 : le contrôle d'accès obligatoire avec AppArmor

5.1 Le concept

AppArmor est le système MAC par défaut de Debian et d'Ubuntu. Il associe à un exécutable un profil qui liste précisément les fichiers et opérations autorisés ; tout le reste est refusé, y compris pour root. AppArmor raisonne sur des chemins, ce qui le rend simple à lire ; SELinux (monde Red Hat) raisonne sur des étiquettes attachées aux fichiers et processus, plus fin mais plus complexe.

Mode	Comportement
complain	Ne bloque rien, mais journalise ce qui serait refusé. Idéal pour construire un profil.
enforce	Applique : bloque et journalise les violations.

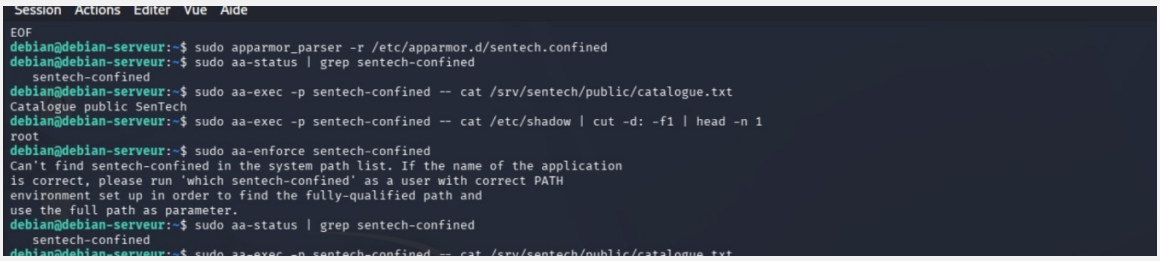
5.2 Observer l'existant

Sur Debian Serveur

```
sudo aa-status
```

CAPTURE 15 : État d'AppArmor

À montrer : La sortie de sudo aa-status (profils chargés, docker-default dans la liste).



5.3 Manipulation : le MAC domine root

Objectif. Écrire un profil qui autorise la lecture de /srv/sentech/public/ mais interdit celle de /etc/shadow, même exécuté en root.

Étape 1, préparer le fichier de démonstration.

Sur Debian Serveur

```
echo "Catalogue public SenTech" | sudo tee /srv/sentech/public/catalogue.txt
sudo chmod 644 /srv/sentech/public/catalogue.txt
ls -l /srv/sentech/public/
```

Avec umask 027, le fichier aurait été créé en 640, illisible pour les autres : le `chmod 644` rend le dossier « public » cohérent avec la partie durcissement fichiers du rapport 1.

Étape 2, créer le profil.

Sur Debian Serveur

```
sudo tee /etc/apparmor.d/sentech.confined > /dev/null <<'EOF'
abi <abi/3.0>,
include <tunables/global>

profile sentech-confined flags=(complain) {
 include <abstractions/base>

 # Lecture autorisee : uniquement le dossier public de SenTech
 /srv/sentech/public/ r,
 /srv/sentech/public/** r,

  # Tout le reste est implicitement REFUSE (aucune autre regle d'acces).
}
EOF
```

Lecture du profil. `abi` fixe la version de la syntaxe ; `include <tunables/global>` charge des variables communes ; `profile sentech-confined flags=(complain)` déclare un profil nommé, d'abord en mode complain ; `include <abstractions/base>` autorise le minimum vital (charger les bibliothèques) sans quoi aucun programme ne démarrerait ; les deux lignes `r,` autorisent la lecture du dossier et de son contenu. Il n'y a aucune règle sur /etc/shadow : c'est le refus implicite.

Étape 3, charger le profil.

Sur Debian Serveur

```
sudo apparmor_parser -r /etc/apparmor.d/sentech.confined
sudo aa-status | grep sentech-confined
```

Le profil apparaît en mode complain. (Si le chargement échoue sur la ligne abi, on remplace abi/3.0 par une version présente dans /etc/apparmor.d/abi/ ou on supprime la ligne.)

Étape 4, test en mode complain (rien n'est bloqué, mais les violations sont notées). On n'affiche que le premier champ de /etc/shadow pour ne pas exposer de hachage de mot de passe dans le rapport.

Sur Debian Serveur

```
sudo aa-exec -p sentech-confined -- cat /srv/sentech/public/catalogue.txt
sudo aa-exec -p sentech-confined -- cat /etc/shadow | cut -d: -f1 | head -n 1
```

Étape 5, passer en enforce et tester.

Sur Debian Serveur

```
sudo aa-enforce sentech-confined
sudo aa-status | grep sentech-confined

sudo aa-exec -p sentech-confined -- cat /srv/sentech/public/catalogue.txt
sudo aa-exec -p sentech-confined -- cat /etc/shadow
```

Résultat attendu

```
Catalogue public SenTech
cat: /etc/shadow: Permission denied
```

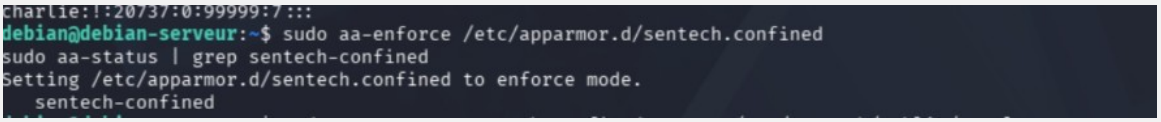
Analyse. La démonstration centrale : la lecture du catalogue marche, celle de /etc/shadow échoue bien que la commande soit lancée avec `sudo`. La politique s'impose au-dessus de l'identité. Si le catalogue lui-même était refusé, il manquerait une règle pour `cat` (par exemple `/usr/bin/cat mr,`) : on la lit dans le journal des refus puis on ajuste le profil, ce qui est la méthode réelle d'un administrateur.

CAPTURE 16 : Profil créé, chargé en complain

À montrer : Le contenu du profil, aa-status montrant sentech-confined en mode complain, puis les deux tests du mode complain.



CAPTURE 17 : Test en enforce : le MAC domine root



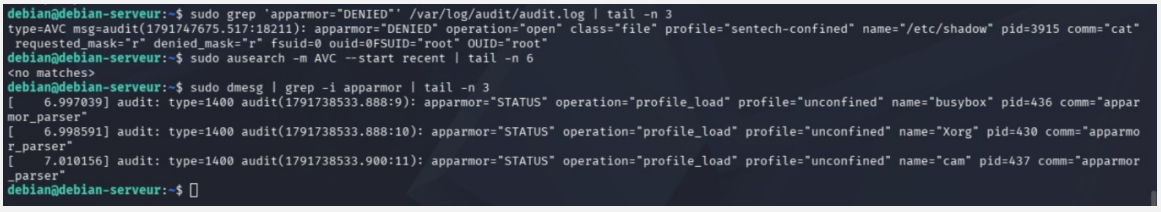
Étape 6, retrouver la trace du refus. Ici, le laboratoire Debian diffère du sujet.

Sur Debian Serveur

```
sudo grep 'apparmor="DENIED"' /var/log/audit/audit.log | tail -n 3
sudo ausearch -m AVC --start recent | tail -n 6
sudo dmesg | grep -i apparmor | tail -n 3
```

Analyse. Le profil sentech-confined a refusé l'opération `open` en lecture sur /etc/shadow, demandée par `cat`, alors que `fsuid=0` : l'identité était root. Le sujet cherche le refus avec `dmesg`, mais Auditd est actif sur notre serveur : le noyau lui envoie les messages d'audit, dont ceux d'AppArmor, au lieu de les écrire dans le journal du noyau. `dmesg` ne montre rien et on croirait, à tort, qu'AppArmor n'a rien bloqué. La bonne source est /var/log/audit/audit.log. C'est un résultat propre à notre laboratoire, qui montre comment nos deux outils (Auditd, AppArmor) coopèrent.

CAPTURE 18 : La trace du refus dans Auditd



Pourquoi le DAC seul n'aurait pas empêché ce processus root de lire /etc/shadow. Le DAC vérifie l'identité et les permissions du fichier ; or root possède la capability CAP_DAC_OVERRIDE, qui lui fait ignorer ces permissions, donc `cat` lancé en root peut lire /etc/shadow malgré le mode 640. AppArmor applique une politique liée au programme (le profil) et non à l'identité : il est consulté en plus du DAC et refuse l'accès, même à root.

Partie 6 : sécurité des conteneurs Docker, assembler les mécanismes

Un conteneur Docker combine les quatre briques, mais ses réglages par défaut sont un compromis, pas un durcissement maximal. Cette partie observe le défaut, puis construit un conteneur durci.

6.1 Observer un conteneur par défaut

Procédure.

Sur Debian Serveur

```
sudo docker run --rm -it alpine sh
```

Dans le conteneur

```
id
grep Cap /proc/self/status
exit
```

Analyse. Le conteneur tourne en root par défaut (uid 0). Docker lui a retiré beaucoup de capabilités (comparer avec `000001fffffffff` de root sur l'hôte) mais lui en laisse quatorze : c'est la valeur `a80425fb` décodée en partie 2. Root dans un conteneur n'est donc pas un root « complet », mais c'est un root tout de même.

CAPTURE 19 : Conteneur par défaut



Profil AppArmor appliqué, puis seccomp par défaut.

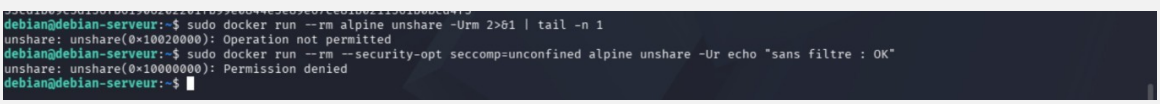
Sur Debian Serveur

```
CID=$(sudo docker run -d --rm alpine sleep 60)
sudo docker inspect "$CID" --format 'AppArmor={{.AppArmorProfile}}'
sudo docker kill "$CID"

sudo docker run --rm alpine unshare -Urm 2>&1 | tail -n 1
sudo docker run --rm --security-opt seccomp=unconfined alpine unshare -Ur echo
"sans filtre : OK"
```

Analyse. La version du sujet avec l'option `-m` (nouveau namespace de montage) peut échouer pour une autre raison : AppArmor (docker-default) refuse l'appel de montage (`cannot change root filesystem propagation: Permission denied`). Ce n'est pas seccomp qui bloque, c'est AppArmor : belle preuve que les protections se superposent. La variante sans `-m` isole seccomp.

CAPTURE 20 : AppArmor et seccomp par défaut



6.2 Construire une image durcie

Procédure. L'application répond « SenTech WebApp - conteneur durci » sur le port 8080 (non privilégié, donc sans capability).

Sur Debian Serveur

```
mkdir -p ~/webapp && cd ~/webapp

cat > app.py <<'EOF'
from http.server import BaseHTTPRequestHandler, HTTPServer

class Handler(BaseHTTPRequestHandler):
   def do_GET(self):
     self.send_response(200)
     self.send_header("Content-Type", "text/plain; charset=utf-8")
     self.end_headers()
     self.wfile.write("SenTech WebApp - conteneur durci\n".encode())

if __name__ == "__main__":
   HTTPServer(("0.0.0.0", 8080), Handler).serve_forever()
EOF

cat > Dockerfile <<'EOF'
# Base minimale = moins de paquets = moins de vulnerabilites
FROM python:3.12-slim

# Creer un utilisateur non privilegie dedie
RUN useradd --create-home --uid 10001 appuser

WORKDIR /app
COPY --chown=appuser:appuser app.py .

# Le conteneur ne tournera PAS en root
USER appuser
EXPOSE 8080
CMD ["python", "app.py"]
EOF

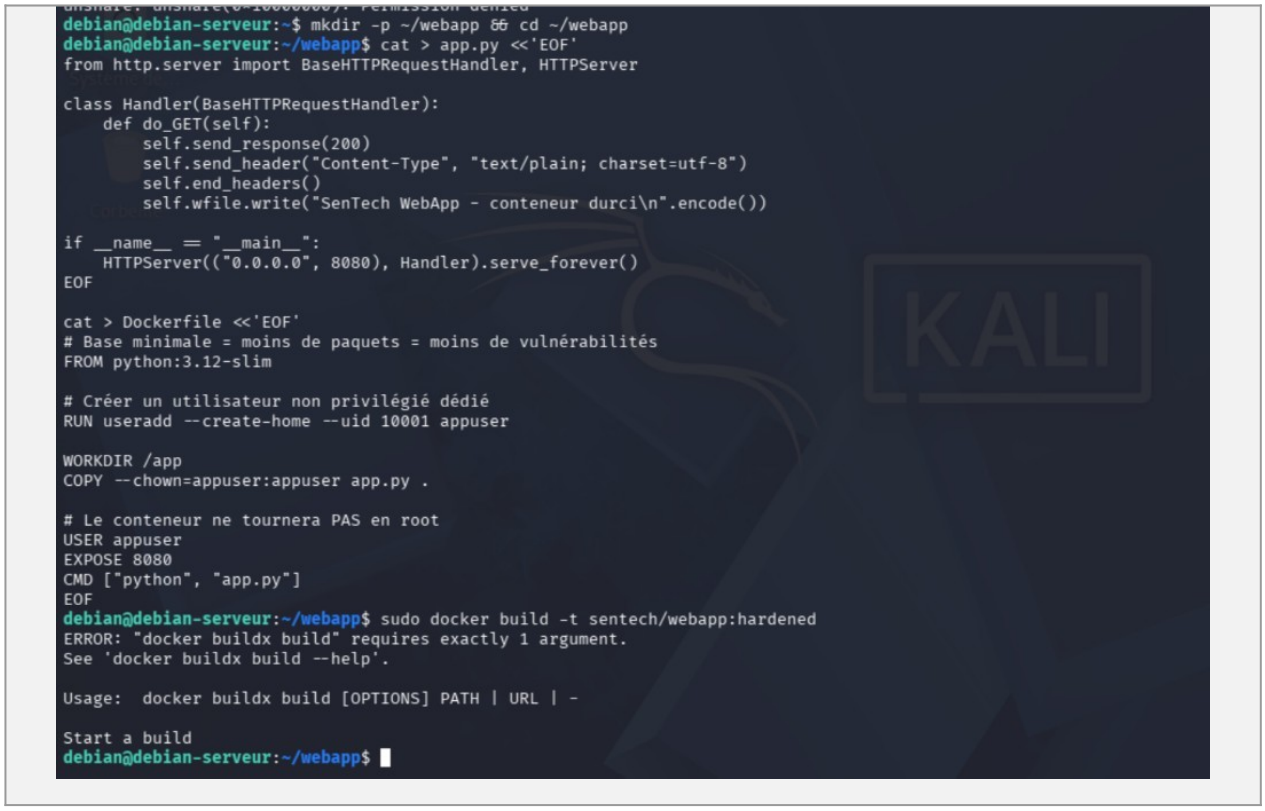
sudo docker build -t sentech/webapp:hardened .
```

Lecture du Dockerfile. `FROM` choisit une base Debian allégée (slim) ; `RUN useradd` crée l'utilisateur 10001 ; `WORKDIR` fixe le dossier de travail ; `COPY` copie l'application ; `USER` fait passer tout ce qui suit (y compris le processus final) sous cet utilisateur ; `CMD` est la commande de démarrage.

DIFFICULTÉ **Le umask 027 empêche le conteneur de démarrer (correction : COPY --chown)**

Avec umask 027, app.py est créé en 640. Le `COPY` du sujet le place dans l'image appartenant à root avec ces permissions ; or le conteneur s'exécute en appuser (uid 10001), qui ne peut pas le lire : le conteneur s'arrête aussitôt et `docker logs` affiche `python: can't open file '/app/app.py': [Errno 13] Permission denied`. La correction est `COPY --chown=appuser:appuser` (ou un chmod 644 avant la construction). Sur la machine du professeur, avec le umask 022 d'Ubuntu, le problème n'apparaît pas : c'est notre durcissement qui le révèle.

CAPTURE 21 : Construction de l'image



6.3 Écrire un profil seccomp personnalisé

Sur Debian Serveur

```
cat > ~/webapp/seccomp-sentech.json <<'EOF'
{
  "defaultAction": "SCMP_ACT_ALLOW",
  "syscalls": [
    {
      "names": ["chmod","fchmod","fchmodat","chown","fchown","fchownat",
            "mount","umount2","ptrace","unshare","setns","keyctl"],
      "action": "SCMP_ACT_ERRNO",
      "errnoRet": 1
    }
  ]
}
EOF

jq . ~/webapp/seccomp-sentech.json > /dev/null && echo "JSON valide"
```

Lecture. `defaultAction ALLOW` signifie « tout est permis par défaut » ; la liste `names` regroupe les appels interdits (`ERRNO` avec `errnoRet: 1` = « opération non permise »). La dernière ligne vérifie que le JSON est correct (une virgule oubliée rendrait le profil inutilisable).

Précision. Ce profil est plus permissif que celui de Docker : le profil par défaut part de « tout refuser sauf une liste » (liste blanche d'environ 300 appels), plus sûr. Le nôtre part de « tout autoriser sauf une liste » : il sert à comprendre le mécanisme, pas à la production.

CAPTURE 22 : Profil seccomp valide



6.4 Lancer le conteneur durci

DIFFICULTÉ **L'image doit exister avant de lancer le conteneur (reconstruction après restauration d'un instantané)**

`docker run` échoue avec `Unable to find image 'sentech/webapp:hardened' locally` si l'image construite en 6.2 n'est plus présente sur le serveur, en particulier après une restauration d'instantané QEMU/KVM faite entre-temps, Docker repart alors de l'état sauvegardé et ne connaît plus l'image. On vérifie donc toujours sa présence avant de lancer le conteneur, et on la reconstruit au besoin avec les mêmes commandes qu'en 6.2 (`cd ~/webapp`, `docker build -t sentech/webapp:hardened .`) :

Sur Debian Serveur

```
sudo docker images | grep sentech
```

Procédure.

Sur Debian Serveur

```
sudo docker run -d --name webapp \
 --user 10001:10001 \
 --read-only --tmpfs /tmp \
 --cap-drop=ALL \
 --security-opt=no-new-privileges \
 --security-opt seccomp=/home/debian/webapp/seccomp-sentech.json \
 --security-opt apparmor=docker-default \
 --pids-limit=100 \
 --memory=128m --cpus=0.5 \
 -p 8080:8080 \
 sentech/webapp:hardened

sudo docker ps
curl -s http://localhost:8080
```

Décryptage des options : à quel levier du cours correspond chacune.

Option	Levier	Effet
--user 10001:10001 (et USER)	Capabilités (identité non root)	Le processus n'est plus root, donc n'a aucun pouvoir privilégié effectif
--cap-drop=ALL	Capabilités	Retire les quatorze capabilités restantes
--security-opt seccomp=...	seccomp	Interdit chmod, chown, mount, ptrace, unshare, setns, keyctl

--security-opt apparmor=docker-default	MAC (AppArmor)	Confine les accès fichiers et les opérations sensibles
--read-only + --tmpfs /tmp	Namespace de montage (MNT)	Racine en lecture seule ; seul /tmp (en mémoire) est inscriptible
--security-opt=no-new-privileges	Capabilités	Interdit de gagner des pouvoirs plus tard (SUID, capability de fichier)
--pids-limit=100, --memory=128m, --cpus=0.5	cgroups	Plafonne processus, mémoire et processeur

CAPTURE 23 : Conteneur durci en fonctionnement

À montrer : sudo docker ps (état Up) et la réponse de curl.



6.5 Tester le durcissement

Les trois tests du sujet.

Sur Debian Serveur

```
sudo docker exec webapp id
sudo docker exec webapp chmod 777 /app 2>&1
sudo docker exec webapp sh -c 'echo test > /app/intrus.txt' 2>&1
```

Limite du test n° 2. /app appartient à root (créé par WORKDIR) : même sans seccomp, appuser ne pourrait pas faire `chmod` sur un dossier qui ne lui appartient pas. Le test ne distingue donc pas seccomp de simples permissions. Les preuves A à E ci-dessous prouvent vraiment l'effet de chaque protection.

Sur Debian Serveur

```
# A : sur SON PROPRE fichier, chmod est quand meme refuse (c'est seccomp)
sudo docker exec webapp sh -c 'touch /tmp/mon_fichier && chmod 777
/tmp/mon_fichier' 2>&1

# B (temoin) : meme test SANS le profil seccomp personnalise -> ca reussit
sudo docker run --rm --user 10001:10001 --read-only --tmpfs /tmp python:3.12-
slim \
 sh -c 'touch /tmp/mon_fichier && chmod 777 /tmp/mon_fichier && echo "chmod OK
sans le profil"'

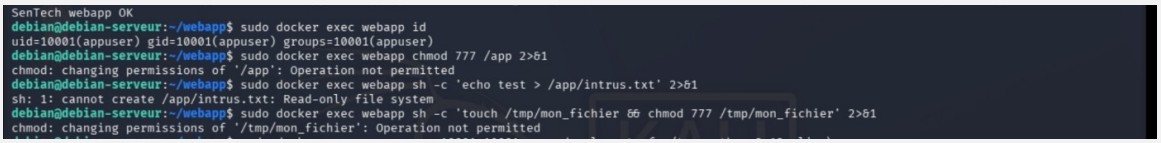
# C : /tmp, lui, reste inscriptible
sudo docker exec webapp sh -c 'echo ok > /tmp/ok.txt && cat /tmp/ok.txt'

# D : plus aucune capability, et pas d'elevation possible
sudo docker exec webapp grep -E 'CapEff|NoNewPrivs' /proc/1/status

# E : les limites et les protections vues par Docker
```

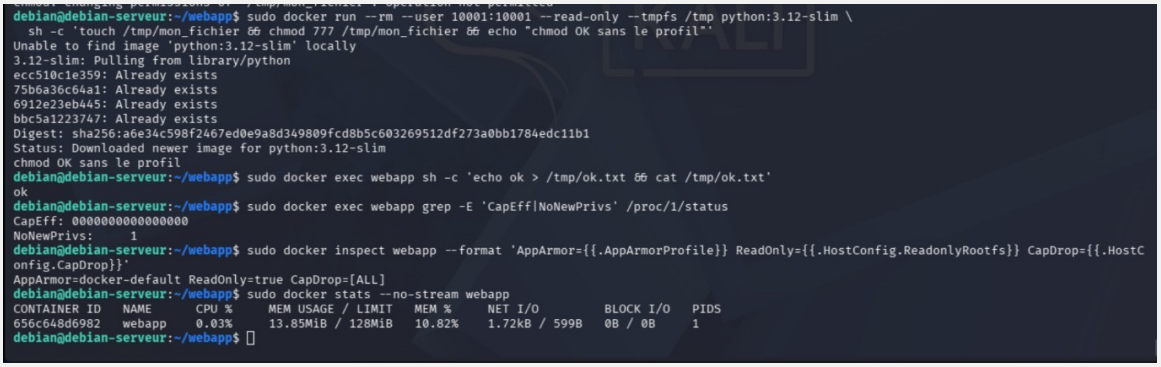
```
sudo docker inspect webapp --format 'AppArmor={{.AppArmorProfile}}
ReadOnly={{.HostConfig.ReadonlyRootfs}} CapDrop={{.HostConfig.CapDrop}}'
sudo docker stats --no-stream webapp
```

CAPTURE 24 : Les trois tests du sujet



CAPTURE 25 : Preuves A à E

À montrer : Les cinq sorties : chmod refusé sur son propre fichier, témoin B réussi, ok, CapEff nul et NoNewPrivs: 1, puis docker inspect et docker stats.



6.6 Constat de sécurité : Docker contourne le pare-feu INPUT

Procédure, depuis Kali.

Sur Kali Linux

```
curl -s --max-time 5 http://192.168.10.10:8080
nmap -Pn -p 8080,80,2222 192.168.10.10
```

Constat. La page s'affiche et le port 8080 apparaît « open », alors que notre politique INPUT est en DROP et qu'aucune règle n'autorise ce port. Les ports 80 et 2222 se comportent comme au rapport 1 (80 filtré, 2222 ouvert).

Explication. Quand on publie un port (`-p 8080:8080`), Docker ajoute dans la table nat une règle qui redirige (DNAT) les paquets vers l'adresse interne du conteneur (172.17.0.x). Le paquet n'est plus « pour cette machine » mais « à faire suivre » : il traverse la chaîne FORWARD, où Docker a inséré ses propres règles d'acceptation avant les nôtres. Notre politique INPUT DROP ne le voit jamais. C'est un défaut classique : ufw ou iptables donnent l'impression de protéger un port que Docker a pourtant ouvert. C'est une illustration du principe « un durcissement n'est efficace que s'il est vérifié ».

CAPTURE 26 : Docker contourne INPUT



Remède : publier uniquement sur le loopback. Le conteneur est recréé avec la seule modification du port (c'est cette version qui est conservée).

Sur Debian Serveur

```
sudo docker rm -f webapp
sudo docker run -d --name webapp \
 --user 10001:10001 \
 --read-only --tmpfs /tmp \
 --cap-drop=ALL \
 --security-opt=no-new-privileges \
 --security-opt seccomp=/home/debian/webapp/seccomp-sentech.json \
 --security-opt apparmor=docker-default \
 --pids-limit=100 \
 --memory=128m --cpus=0.5 \
 -p 127.0.0.1:8080:8080 \
 sentech/webapp:hardened

curl -s http://localhost:8080
```

Sur Kali Linux

```
curl -s --max-time 5 http://192.168.10.10:8080 ; echo "code = $?"
```

CAPTURE 27 : Remède appliqué



Partie 7 : hygiène des images, analyse avec Trivy

Objectif. Une image est un empilement de couches contenant un système de fichiers complet ; chaque composant peut porter des vulnérabilités connues (CVE). Trivy compare le contenu d'une image à des bases de vulnérabilités et classe les résultats par gravité (CRITICAL, HIGH, MEDIUM, LOW). C'est, pour les images, l'équivalent de ce qu'OpenVAS a fait pour le serveur au rapport 1.

7.1 Installation

Sur Debian Serveur

```
sudo apt install -y wget gnupg

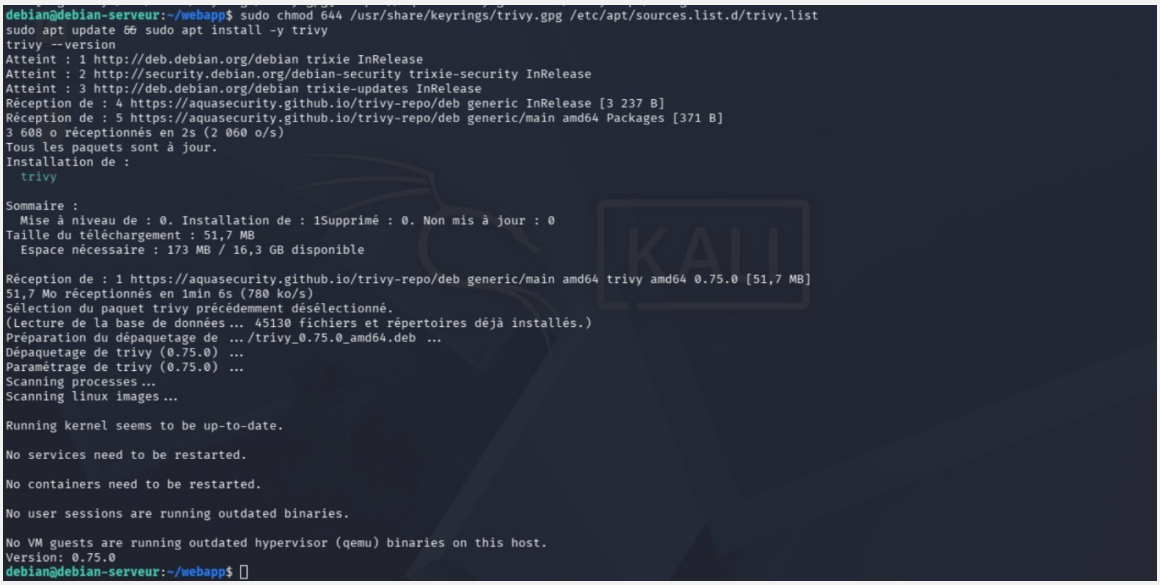
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key \
 | sudo gpg --dearmor -o /usr/share/keyrings/trivy.gpg
```

```
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg]
https://aquasecurity.github.io/trivy-repo/deb generic main" \
 | sudo tee /etc/apt/sources.list.d/trivy.list

sudo chmod 644 /usr/share/keyrings/trivy.gpg /etc/apt/sources.list.d/trivy.list
sudo apt update && sudo apt install -y trivy
trivy --version
```

Principe. On télécharge la clé de signature du dépôt, on la convertit au format attendu par apt (--dearmor), on déclare le dépôt en imposant que ses paquets soient signés par cette clé (signed-by), puis on installe trivy.

CAPTURE 28 : Trivy installé



7.2 Comparer une image ancienne et une image à jour

Sur Debian Serveur

```
sudo trivy image --scanners vuln --severity HIGH,CRITICAL nginx:1.14 | tail -n
40
sudo trivy image --scanners vuln --severity HIGH,CRITICAL nginx:stable | tail -n
40
```

Le premier lancement télécharge la base de vulnérabilités puis chaque image : plusieurs minutes. --scanners vuln limite l'analyse aux vulnérabilités (le scan de secrets est plus lourd en mémoire). Trivy se lance avec sudo car l'accès au moteur Docker (/var/run/docker.sock) est réservé à root ; sans sudo, l'image locale sentech/webapp:hardened ne serait jamais trouvée.

Pour remplir le tableau de priorisation sans compter à la main, la fonction suivante résume une image (nombre par gravité et vulnérabilité la plus grave avec son score) :

Sur Debian Serveur

```
tsum() {
  sudo trivy image -q --scanners vuln --severity HIGH,CRITICAL -f json "$1" >
~/trivy_tmp.json
  echo "=== $1"
  jq -r '[.Results[]?.Vulnerabilities[]?] | group_by(.Severity)[] | "\(.
[0].Severity): \(length)"' ~/trivy_tmp.json
  jq -r '[.Results[]?.Vulnerabilities[]? |
{id:.VulnerabilityID,pkg:.PkgName,score:(([.CVSS[]?.V3Score]|max) // 0)}] |
sort_by(-.score) | .[0] // empty | "Plus grave: \(.id) (\(.pkg)) score \
```

```
(.score)"' ~/trivy_tmp.json
}

tsum nginx:1.14
tsum nginx:stable
tsum sentech/webapp:hardened
```

La redirection `> ~/trivy_tmp.json` est faite par notre shell : le fichier nous appartient et jq peut le lire (avec l'option -o de Trivy sous sudo, le fichier serait à root en 640 à cause du umask 027).

CAPTURE 29 : Analyse Trivy des deux images nginx



CAPTURE 30 : Résumé par la fonction tsum

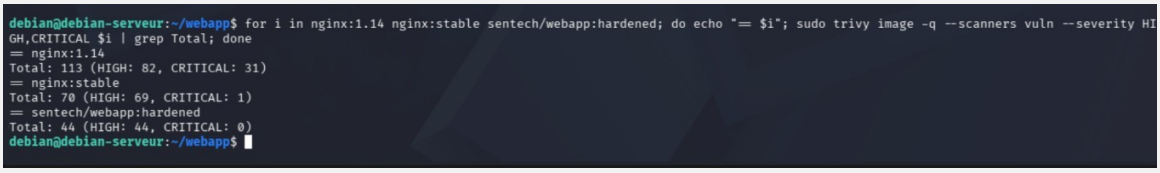


Tableau de priorisation. À compléter avec nos chiffres (ceux de tsum) et la date du scan, car les bases évoluent chaque jour. Ordre de grandeur attendu : nginx:1.14 en centaines de HIGH/CRITICAL, nginx:stable et webapp en quelques unités à quelques dizaines.

Image	CRITICAL	HIGH	CVE la plus grave (ID, score)	Action recommandée
nginx:1.14	31	82	CVE-2022-1664 (dpkg) score 9.8	Ne pas déployer : image en fin de vie, migrer vers une version maintenue
nginx:stable	1	69	CVE-2026-6653 (libxml2) score 9.8	Acceptable ; planifier une mise à jour régulière
sentech/ webapp:hardened	0	44	CVE-2026-16742 (libsystemd0) score 8.8	Mettre à jour la base python:3.12-slim et rescanner à chaque

				construction

7.3 Analyser notre image et nos fichiers de configuration

Sur Debian Serveur

```
sudo trivy image --scanners vuln --severity HIGH,CRITICAL
sentech/webapp:hardened
sudo trivy config ~/webapp
```

La seconde commande ne regarde pas les paquets : elle lit le Dockerfile et signale des mauvaises pratiques. Résultat typique : AVD-DS-0026 « No HEALTHCHECK defined ». On ne voit pas « Image user should not be root » : c'est le travail du USER appuser.

Correction. Ajouter dans le Dockerfile, juste avant CMD, une sonde de santé (sans curl, absent de l'image), puis reconstruire l'image et relancer le conteneur :

Dans ~/webapp/Dockerfile (avant CMD)

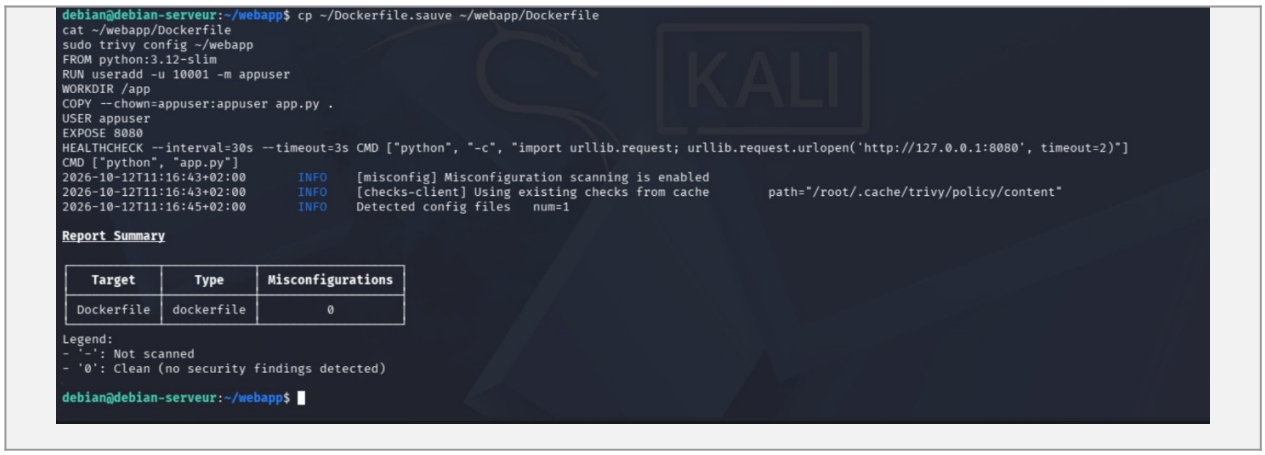
```
HEALTHCHECK --interval=30s --timeout=3s CMD ["python", "-c", "import
urllib.request; urllib.request.urlopen('http://127.0.0.1:8080', timeout=2)"]
```

La sonde interroge la page toutes les 30 secondes ; Docker marque le conteneur « unhealthy » si elle échoue, ce qui permet de détecter un conteneur bloqué. Après reconstruction, `sudo trivy config ~/webapp` ne signale plus l'alerte.

CAPTURE 31 : trivy config avant correction



CAPTURE 32 : trivy config après correction



Partie 8 : comprendre pour prévenir, l'évasion de conteneur

Cette partie reproduit une mauvaise configuration pour en mesurer le danger. Elle se déroule uniquement dans le laboratoire, avec des configurations publiquement documentées comme dangereuses ; la finalité est de savoir les repérer et les interdire.

8.1 Le concept

Un conteneur partage le noyau de l'hôte : son isolation ne tient qu'aux mécanismes des parties 2 à 5. Casser un seul d'entre eux peut suffire à s'évader.

Mauvaise configuration	Pourquoi c'est dangereux
--privileged	Désactive presque toutes les protections : le conteneur est quasiment root sur l'hôte
Monter le système hôte (-v /:/host)	Le conteneur lit et écrit les fichiers de l'hôte
Monter /var/run/docker.sock	Le conteneur pilote le moteur Docker et peut créer un conteneur privilégié
--cap-add=SYS_ADMIN	Rouvre la porte à de nombreuses techniques d'évasion
Tourner en root dans le conteneur	Amplifie l'impact de toute faille

8.2 Démonstration contrôlée : lire les fichiers de l'hôte

Sur Debian Serveur

```
# CONFIGURATION DANGEREUSE - a ne JAMAIS reproduire en production
sudo docker run --rm -it -v /:/host:ro alpine sh
```

Dans le conteneur

```
id
cat /host/etc/hostname
cut -d: -f1 /host/etc/shadow | head -n 5
ls -la /host/srv/sentech/finance/
exit
```

Analyse. Le montage en lecture seule suffit à montrer le problème. `cut -d: -f1` n'affiche que les noms de comptes (pas les hachages de mots de passe, qui n'ont pas à figurer dans un rapport) et prouve la même chose : le fichier est lisible. Le contenu du dossier finance (rapport 1) est visible alors que le conteneur n'a rien

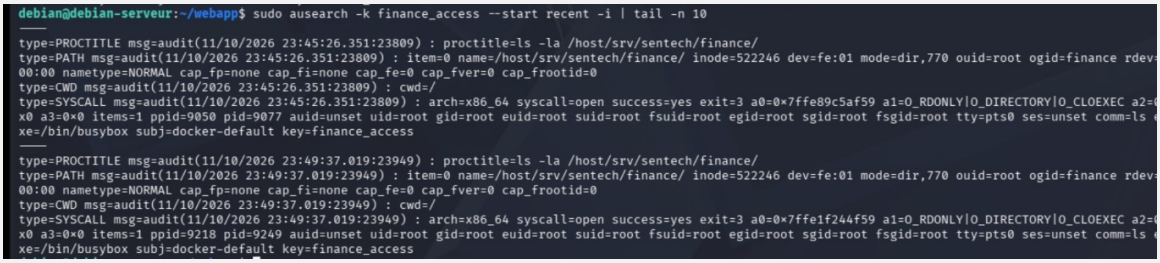
à y faire. Constat : l'isolation a été volontairement percée par -v /:/host ; c'est précisément ce qu'un attaquant cherche après avoir compromis une application conteneurisée.

La règle Auditd finance_access du rapport 1 a vu cet accès : `sudo ausearch -k finance_access --start recent -i | tail -n 10`. Le conteneur et l'hôte partagent le même noyau ; l'isolation visuelle n'empêche pas la traçabilité, ce qui justifie de garder Auditd en plus de Falco.

CAPTURE 33 : Évasion par montage de l'hôte



CAPTURE 34 : Auditd a vu l'accès du conteneur



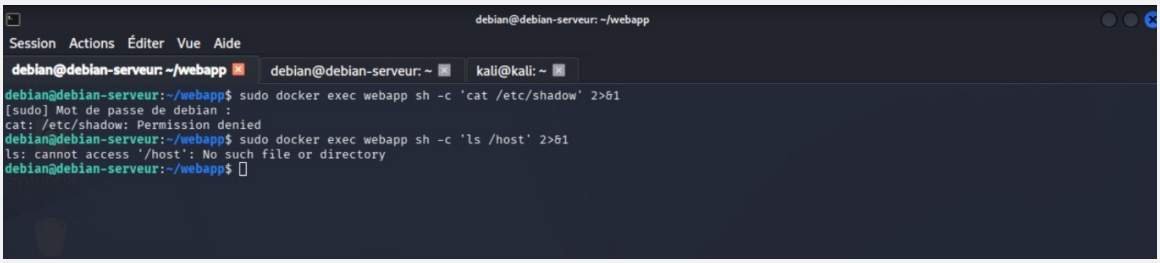
8.3 Vérifier que le conteneur durci résiste

Sur Debian Serveur

```
sudo docker exec webapp sh -c 'cat /etc/shadow' 2>&1
sudo docker exec webapp sh -c 'ls /host' 2>&1
```

Analyse. Le conteneur durci ne monte aucun chemin de l'hôte, tourne en non-root et sans capability. Le sujet annonce que son /etc/shadow « ne contient pas les hachages de l'hôte » ; dans notre image (python:3.12-slim) le fichier appartient à root en 640 : appuser reçoit simplement Permission denied, un meilleur résultat encore.

CAPTURE 35 : Le conteneur durci résiste



8.4 Remédiations à retenir

- Jamais --privileged ni de montage de docker.sock dans un conteneur applicatif.

- Ne monter aucun chemin sensible de l'hôte ; si un montage est indispensable, le limiter au dossier précis et le passer en lecture seule (:ro).
- Tourner en non-root (USER, --user) et en --cap-drop=ALL.
- Activer la remise en correspondance des UID (user namespaces) au niveau du démon ({ "userns-remap": "default" } dans /etc/docker/daemon.json) : root dans le conteneur correspond alors à un utilisateur sans privilège sur l'hôte.

> **ATTENTION** **userns-remap : à citer, pas à activer pendant le TP**
>
> L'activer change le dossier de stockage de Docker : toutes les images et tous les conteneurs existants disparaissent de la liste, et webapp devrait être reconstruit. Un daemon.json mal écrit empêche aussi Docker de démarrer. Cette bonne pratique est donc citée sans être appliquée.

Mécanismes contournés par -v /:/host. Il contourne le namespace de montage : le conteneur ne voit plus un système de fichiers qui lui est propre mais celui de l'hôte. Il s'appuie sur le fait que le processus est root et conserve CAP_DAC_OVERRIDE (jeu par défaut de Docker), donc il ignore les permissions fichiers. Le profil docker-default ne l'empêche pas, car il n'interdit pas la lecture d'un chemin monté volontairement. seccomp n'est pas en cause : lire un fichier n'utilise que des appels système banals (open, read).

Trois règles d'or pour la documentation interne de SenTech.

7. Aucun conteneur applicatif ne tourne en --privileged ni avec docker.sock monté.
8. Aucun chemin sensible de l'hôte (/, /etc, /srv/sentech/finance) n'est monté ; tout montage nécessaire est limité au dossier précis et en lecture seule.
9. Tout conteneur tourne sous un utilisateur non root avec --cap-drop=ALL, no-new-privileges, un profil seccomp et AppArmor, et ses ressources plafonnées.

Partie 9 : détecter l'intrusion à l'exécution, Falco

Objectif. Le durcissement réduit le risque mais ne le supprime jamais : il faut aussi détecter les comportements suspects en temps réel. Falco observe les événements du noyau (par eBPF) et les compare à des règles ; il alerte, par exemple, quand un shell est ouvert dans un conteneur ou qu'un fichier sensible est lu. Falco est à l'exécution ce qu'Auditd est au journal : une sonde. Auditd surveille ce que nous avons listé ; Falco apporte des règles toutes faites, conscientes des conteneurs.

9.1 Installation

Sur Debian Serveur

```
ls /sys/kernel/btf/vmlinux

curl -fsSL https://falco.org/repo/falcosecurity-packages.asc \
 | sudo gpg --dearmor -o /usr/share/keyrings/falco-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/falco-archive-keyring.gpg]
https://download.falco.org/packages/deb stable main" \
 | sudo tee /etc/apt/sources.list.d/falcosecurity.list

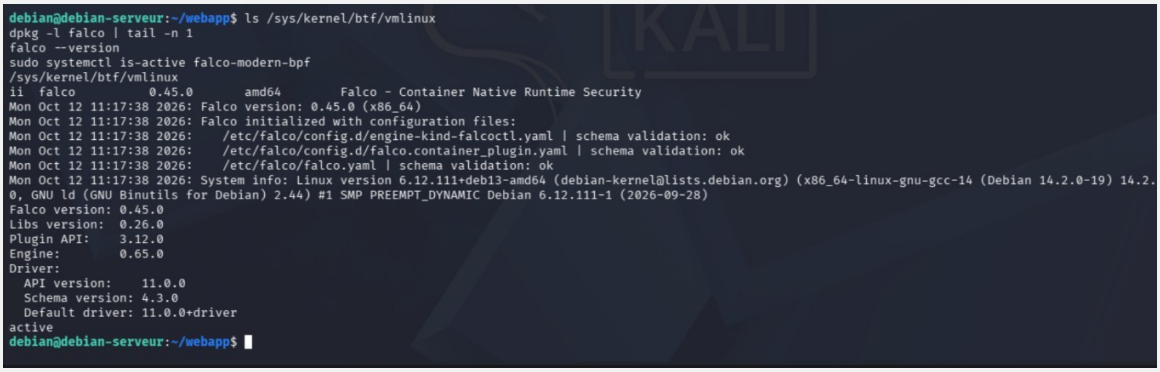
sudo chmod 644 /usr/share/keyrings/falco-archive-keyring.gpg
/etc/apt/sources.list.d/falcosecurity.list
sudo apt update
sudo env FALCO_FRONTEND=noninteractive FALCO_DRIVER_CHOICE=modern_ebpf apt
install -y falco
```

Principe. La première commande vérifie que le noyau fournit ce dont Falco a besoin (eBPF moderne, BTF). Les deux variables d'environnement évitent les questions interactives et choisissent le pilote eBPF moderne.

> DIFFICULTÉ **Ce que le sujet demande en trop, et ce qui démarre tout seul**
>
> Les paquets dkms et linux-headers servent à compiler un pilote noyau (mode kmod) : inutiles avec eBPF moderne, ils consommeraient de l'espace sur un disque de 20 Go. Le `chmod 644` est nécessaire pour la même raison qu'avec Trivy (umask 027). Falco démarre déjà après l'installation (service falco-modern-bpf) : lancer ensuite `sudo falco` à la main ferait tourner deux Falco et dupliquerait les alertes ; on arrête donc d'abord le service (`sudo systemctl stop 'falco*'`).

CAPTURE 36 : Falco installé

À montrer : ls /sys/kernel/btf/vmlinux et la fin de l'installation du paquet falco.



9.2 Déclencher et observer des alertes

Deux fenêtres sur le serveur sont nécessaires (une seule connexion SSH supplémentaire, sous la limite des cinq par minute).

Fenêtre A, lancer Falco au premier plan.

Sur Debian Serveur (fenêtre A)

```
sudo systemctl stop 'falco*'
sudo falco
```

Falco affiche « Loading rules from file /etc/falco/falco_rules.yaml » puis « Opening 'syscall' source with modern BPF probe », puis la fenêtre « se fige » : il attend des événements.

Fenêtre B, provoquer deux comportements surveillés.

Sur Debian Serveur (fenêtre B)

```
sudo docker exec -it webapp sh -c 'echo test'
sudo cat /etc/shadow > /dev/null
```

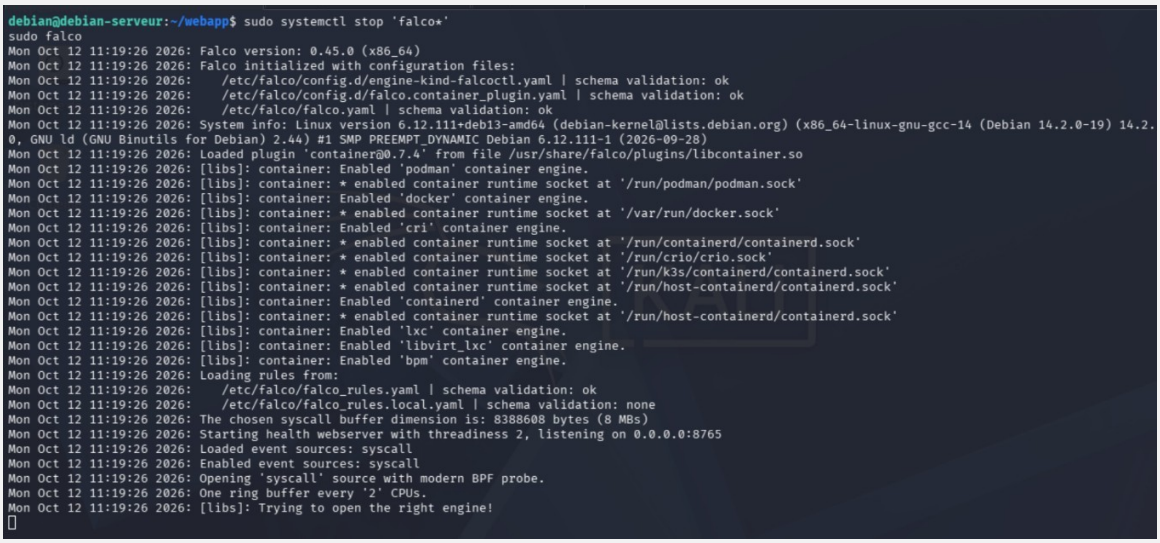
Résultat attendu

```
10:41:03.127: Notice A shell was spawned in a container with an attached
terminal | evt_type=execve user=appuser ... command=sh -c echo test
terminal=34816 ... container_name=webapp ...
10:41:09.455: Warning Sensitive file opened for reading by non-trusted program |
file=/etc/shadow gparent=sudo ... command=cat /etc/shadow ...
```

On arrête Falco par Ctrl+C dans la fenêtre A ; il affiche un récapitulatif du nombre d'événements traités.

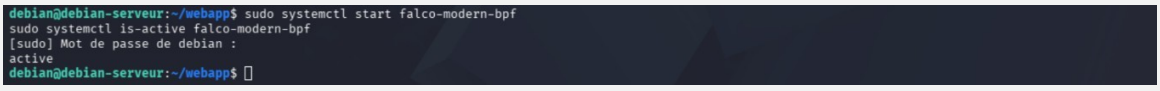
Remarques. L'option -it est indispensable : la règle exige un terminal attaché. Les règles et leur niveau d'activation changent selon la version de Falco : `sudo grep -c 'Terminal shell in container' /etc/falco/falco_rules.yaml` permet de vérifier la présence de la règle ; sinon on utilise l'alerte réellement affichée et on l'indique.

CAPTURE 37 : Falco démarré



CAPTURE 38 : Les deux alertes Falco

À montrer : Dans la fenêtre A : l'alerte « A shell was spawned in a container » (conteneur webapp) et l'alerte « Sensitive file opened for reading » (/etc/shadow), avec leurs horodatages.



À quelle étape d'une attaque correspond chaque alerte ?

- **Shell dans un conteneur :** phase d'exécution (ATT&CK : tactique Exécution, T1059.004 Unix Shell et T1609 Container Administration Command). L'attaquant vient de prendre pied dans le conteneur et cherche à y agir ; c'est aussi le signe d'un accès initial réussi si l'on n'a pas vu l'entrée.
- **Lecture de /etc/shadow :** vol d'identifiants (tactique Accès aux identifiants, T1003.008). L'attaquant cherche des mots de passe pour étendre son accès.

Un comportement supplémentaire à détecter sur un serveur SenTech : une écriture dans /etc/cron.d/ ou la création d'un compte (persistance), mais aussi l'exécution d'un programme depuis /tmp, la création d'un fichier SUID, une connexion sortante inhabituelle d'un conteneur, tout accès à /srv/sentech/finance par un programme non autorisé, ou le montage de docker.sock dans un conteneur.

Partie 10 : synthèse, checklist et fiche finale du TP 1

10.1 La checklist de durcissement d'un conteneur

Trois points ont été révélés par notre laboratoire et ne figurent pas dans le sujet : publier les ports sur 127.0.0.1 ou filtrer dans DOCKER-USER (constat 6.6) ; contrôler les permissions des fichiers copiés dans l'image quand l'hôte a un umask strict (COPY --chown) ; surveiller aussi avec Auditd, qui voit les actions des conteneurs puisque le noyau est partagé.

10.2 Fiche de synthèse du TP 1

Mesure	Levier	Preuve
Port 80 sans root	Capabilités	Partie 2.3 : CapEff: 400
Appels mount interdits	seccomp	Partie 3.2 : permission denied, code 32
Cage de processus	Namespaces	Partie 4.2 : $$ vaut 1
Plafond mémoire 64 Mo	cgroups	Partie 4.3 : OOM-kill
/etc/shadow interdit à root	MAC (AppArmor)	Partie 5.3 : apparmor="DENIED"
Conteneur durci	Les quatre	Partie 6.5 : preuves A à E
Images saines	Hygiène	Partie 7 : tableau Trivy
Évasion neutralisée	Les quatre	Partie 8.3
Détection à l'exécution	Falco	Partie 9.2 : deux alertes

Conclusion du TP 1. Un conteneur n'est qu'un processus entouré de mécanismes du noyau. Aucun n'est suffisant seul : c'est leur combinaison (moindre privilège, filtrage des appels système, cloisonnement, plafonnement et politique obligatoire), complétée par la détection à l'exécution, qui protège l'hôte. Sur notre laboratoire, trois points particuliers ont été identifiés : Docker contourne le pare-feu INPUT, le umask strict exige des chown explicites, et Auditd reçoit les journaux AppArmor.